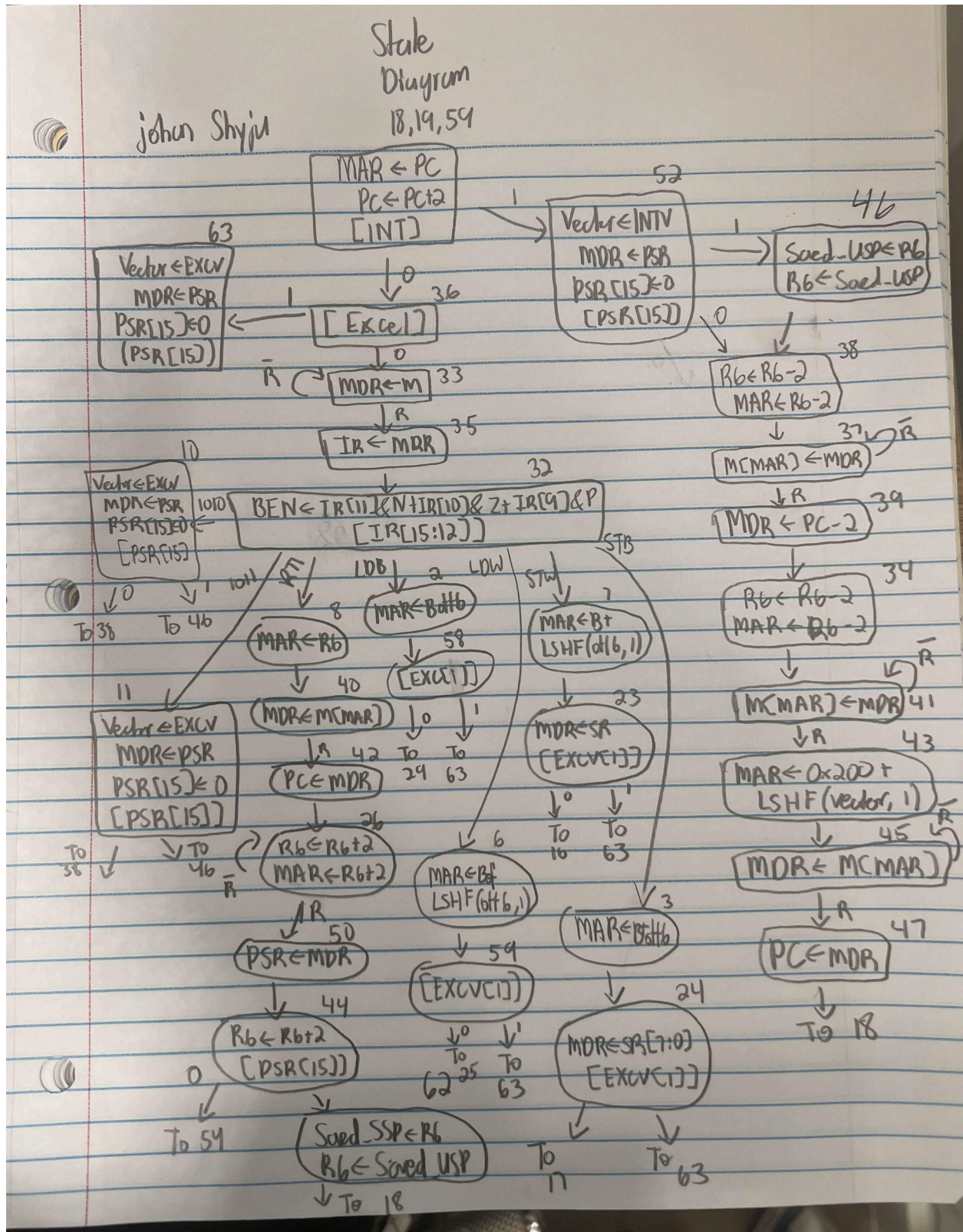




The state machine was changed to support both interrupts and exceptions when the machine is executing instructions. When the machine loads the Memory Address Register to get the instruction, it checks if an

interrupt signal is present. This happens in states 18, 19, and 54. If the machine finds an interrupt, it moves to a series of states that handle the interrupt. These states are 52, 46, 38, 37, 39, 34, 41, 43, 45, and 47. The machine starts by loading the vector into the vector register. This is done in state 52. Then it checks if the machine needs to switch to supervisor mode. If it does, the machine saves the user stack pointer and loads the system stack pointer. This is done in states 52 and 46. Next, the machine saves the program counter and program status register on the system stack. This happens in states 38, 37, 39, 34, and 41. The machine does this so it can start executing the service routine correctly. Finally, the machine updates the program counter with the starting address of the interrupt service routine. This is done in states 43, 45, and 47. The machine also checks for exceptions in state 36. It checks if the program counter is trying to access a protected memory area while in user mode. It also checks if the program counter is not aligned correctly. This can happen with a jump or a jump to a register. If either of these conditions is true, the machine moves to the exception handling sequence. This sequence starts at state 63. The exception handling sequence is similar to the interrupt handling sequence. The main difference is that the vector register is loaded with the exception vector of the interrupt vector. The machine also checks for exceptions before it accesses memory. It does this in states 58, 59, 23, and 24. These states correspond to loading a byte, loading a word, storing a word, and storing a byte. The machine handles opcode exceptions in states 10 and 11. In these states, the machine sets the exception flag. Clears the other exception flag. Finally, the machine implements the return from instruction in states 8, 40, 42, 26, 48, 50, 44, and 62. This instruction restores the program counter and program status register. It does this by removing them from the stack. If the machine is returning to user mode, it swaps the stack pointers. It saves the system stack pointer. Loads the user stack pointer. The state machine does all these things to handle interrupts and exceptions correctly. The state machine is crucial for the machine to function properly. The state machine handles the interrupts and exceptions.

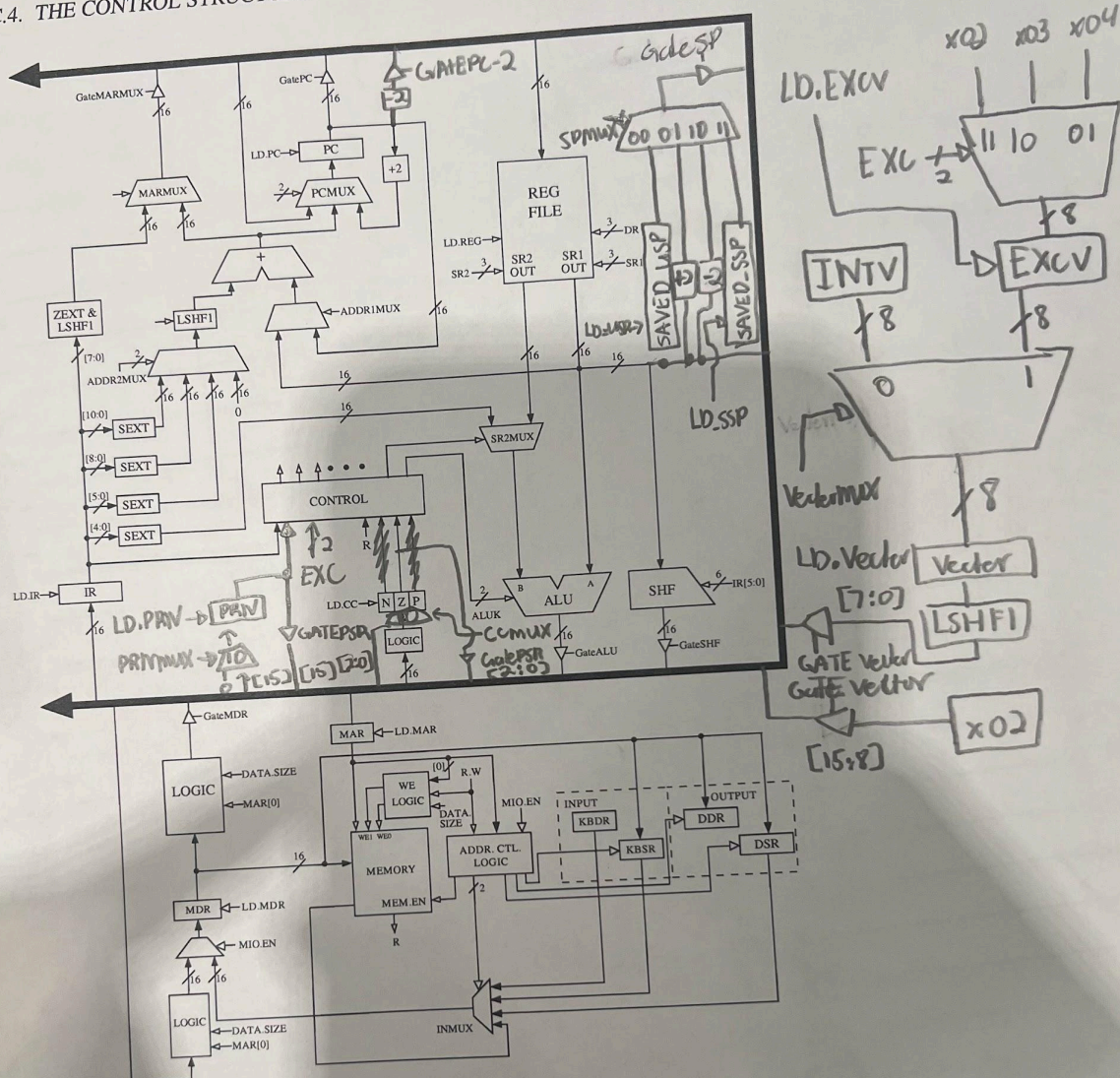


Figure C.3: The LC-3b data path

provide you with the additional flexibility of more states, so we have selected a control store consisting of 2^6 locations.

Changes to the data path: The privilege bit of the PSR was added, and changes (PrivMUX, CCMUX) were made so that the privilege bit and CC could be saved and modified during a context switch. Registers were added to save the user and system stack pointers and allow the stack pointer to be sent to the BUS with a +/-2 offset for push and pop operations (Saved_USP, Saved_SSP, SPMUX). We also allow the PC to be gated on the bus with a -2 offset so that it will be easier to return to the user program

once done with an ISR. Finally, the interrupt and exception vector structures can be seen on the right side, where registers are used to hold the respective vector before being left shifted by 1 and concatenated with x02 and loaded on the bus (INTV, EXCV, EXCVMUX, Vector, VectorMUX). The new control signals added are: LD.Priv, LD.USP, LD.SSP, LD.EXCV, LD.Vector, GatePSR, GateSP, GatePC-2, GateVector, PrivMUX, CCMUX, VectorMUX, and SPMUX (2 bits). The LD signals are used to load the registers with a new value, the gate signals are used to gate the value from a register or the output of a mux to the bus, and the MUX signals are used to select which value should be loaded into a register or to select which value should be sent to the bus (in the case of the SPMUX signal). The EXC signal is a new control signal but is not part of the control store; it is defined by the MAR, PSR[15], and DATA.SIZE. A few changes were made to existing control signals, which are: adding a COND3 bit to allow for exception, interrupt, and privilege check microbranches, and adding an extra bit to both DRMUX and SRMUX so that if they are 10, they will load or source R6 (respectively).

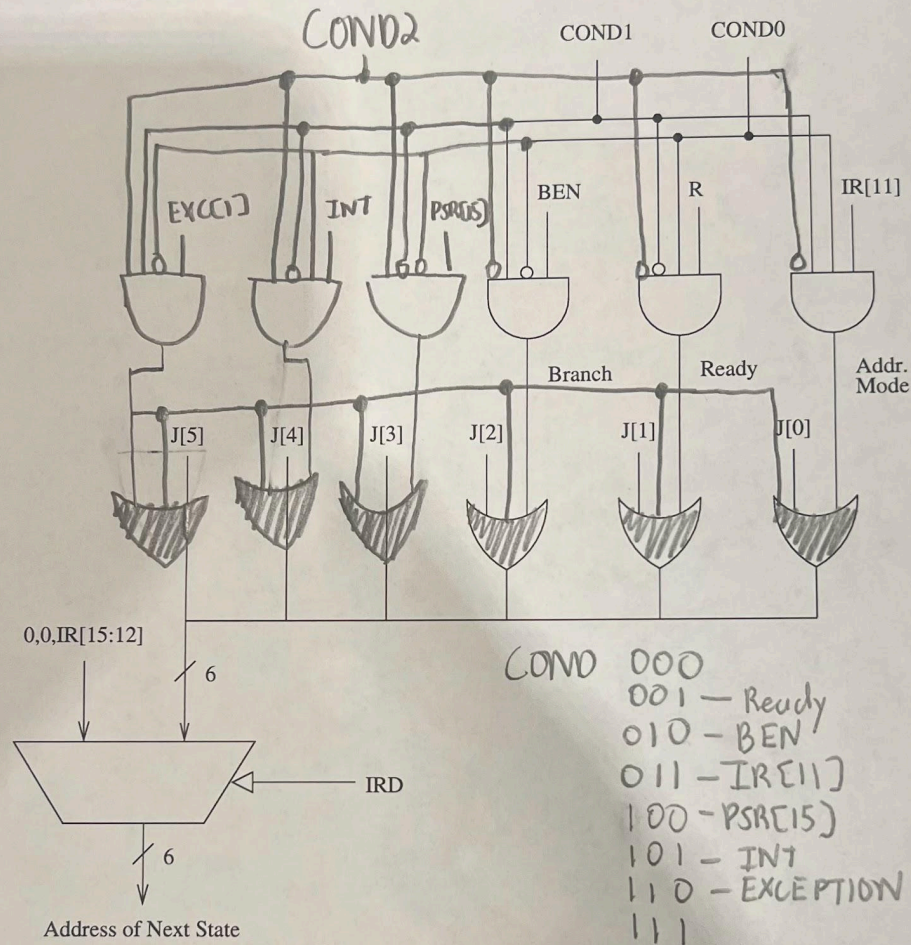


Figure C.5: The microsequencer of the LC-3b base machine

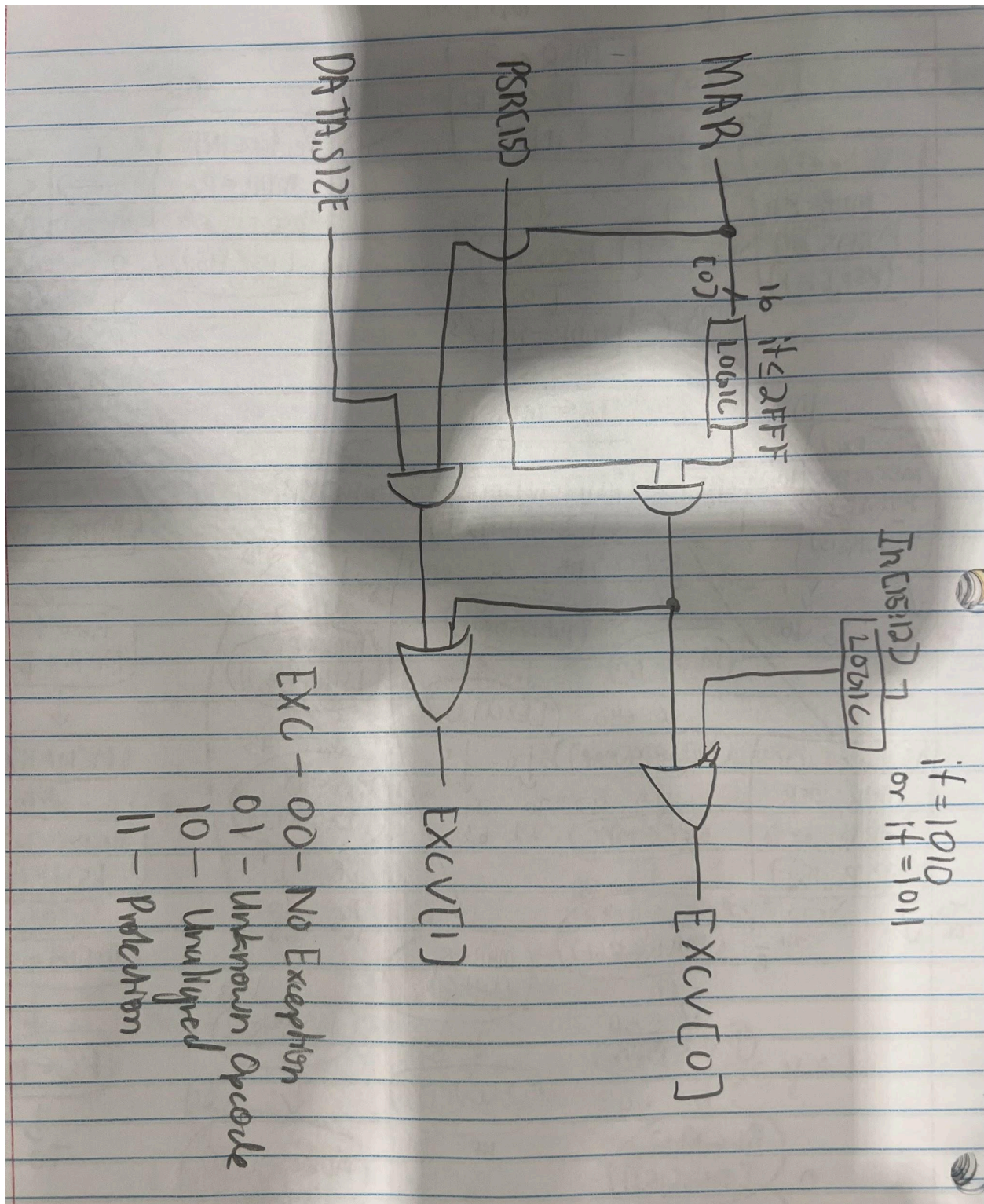
unused opcodes, the microarchitecture would execute a sequence of microinstructions, starting at state 10 or state 11, depending on which illegal opcode was being decoded. In both cases, the sequence of microinstructions would respond to the fact that an instruction with an illegal opcode had been fetched.

Several signals necessary to control the data path and the microsequencer are not among those listed in Tables C.1 and C.2. They are DR, SR1, BEN, and R. Figure C.6 shows the additional logic needed to generate DR, SR1, and BEN.

The remaining signal, R, is a signal generated by the memory in order to allow the

New Microsequencer: The INT signal and PSR[15] are used to microbranch with respect to bits 4 and 3 (respectively) of the state number. However, what is more interesting is the EXC[1] signal, which indicates that an unaligned or protected access exception has occurred. To save states that would be

wasted from only wiring this signal to a single bit for microbranches, I wired all of the 6 J bits to this, so that if EXC[1] is asserted and the COND bits are 110, then we will always branch to state 63.



Defining EXC[0] and EXC[1] bits based on the MAR, PSR[15], and DATA.SIZE.